COMP0005 – ALGORITHMS

Week 2 – 21 Jan 2025

Prof Licia Capra

Dept of Computer Science

OUTLINE

- Topic 1 – Fundamentals
  - Recap of topics 1.1 and 1.2
  - Order of Growth (OOG)
  - Exercises on OOG
- What's next

OUTLINE

- Topic 1 – Fundamentals
  - Recap of topics 1.1 and 1.2
  - Order of Growth (OOG)
  - Exercises on OOG
- What's next

RECAP 1.1

- Topic 1.1: steps to develop an algorithm
  - Model problem
  - Design solution
  - Analyse complexity

RECAP 1.2

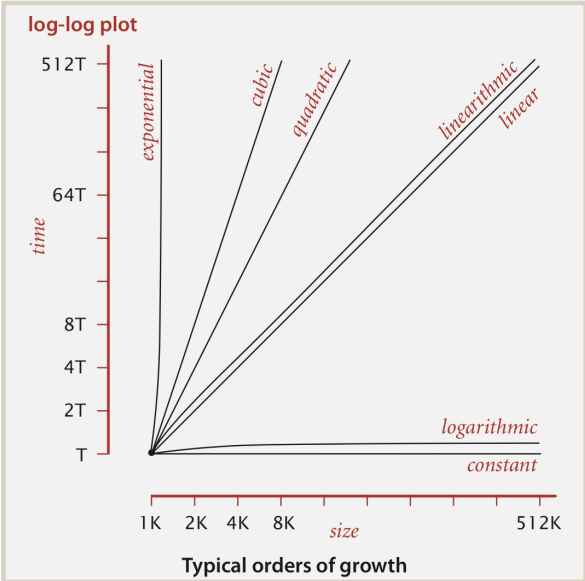
- Topic 1.2: analysis of algorithm
  - Experimental
  - Mathematical
    - Exact vs approximation (~)
  - Order of Growth

order of growth	name	typical code framework	description
1	**constant**	`a = b + c;`	statement
$\log N$	**logarithmic**	`while (N > 1)` `{ N = N / 2; ... }`	divide in half
N	**linear**	`for (int i = 0; i < N; i++)` `{ ... }`	loop
$N \log N$	**linearithmic**	[see mergesort lecture]	divide and conquer

N^2	**quadratic**	`for (int i = 0; i < N; i++)`
		`for (int j = 0; j < N; j++)`
		`{ ... }`
			double loop

N^3	**cubic**	`for (int i = 0; i < N; i++)`
		`for (int j = 0; j < N; j++)`
		`for (int k = 0; k < N; k++)`
		`{ ... }`
			triple loop

2^N	**exponential**	[see combinatorial search lecture]	exhaustive search



Typical orders of growth

OUTLINE

- Topic 1 – Fundamentals
  - Recap of topics 1.1 and 1.2
  - Order of Growth (OOG)
  - Exercises on OOG
- What's next

ON THE ORDER OF GROWTH…

- Topic 1.2: analysis of algorithm
  - Experimental
  - Mathematical
    - Exact vs approximation (~)
  - Order of Growth
    - **Lower bound Ω / upper bound O / tight bound Θ**

ON THE ORDER OF GROWTH…

- Topic 1.2: analysis of algorithm
  - Experimental
  - Mathematical
    - Exact vs approximation (~)
  - Order of Growth
    - **Lower bound Ω / upper bound O / tight bound Θ**

```
sum = 0
for i in range(0,N):
    for j in range(0,N):
        sum = sum + a[i,j]
print(sum)
```

ON THE ORDER OF GROWTH…

- Topic 1.2: analysis of algorithm
  - Experimental
  - Mathematical
    - Exact vs approximation (~)
  - Order of Growth
    - **Lower bound Ω / upper bound O / tight bound Θ**

```
def isPrime(n):
    toTest = 2
    while(toTest < n):
        if (n % toTest == 0):
            return False
        else:
            toTest += 1
    return True
```

ON THE ORDER OF GROWTH…

- Topic 1.2: analysis of algorithm
  - Experimental
  - Mathematical
    - Exact vs approximation (~)
  - Order of Growth
    - **Lower bound Ω / upper bound O / tight bound Θ**

```
def isPrime(n):
    toTest = 2
    while(toTest < n):
        if (n % toTest == 0):
            return False
        else:
            toTest += 1
    return True
```

```
sum=0
if (a[0]<=a[N/2]):
    for i in range(0,N):
        sum = sum + a[i]
else:
    for i in range(N/2,N):
        sum = sum + a[i]
```

RECAP 1.2

- Topic 1.2: analysis of algorithm
  - Experimental
  - Mathematical
    - Exact vs approximation (~)
  - Order of Growth
    - Lower bound Ω / upper bound O / tight bound Θ

CASE ANALYSIS

CASE ANALYSIS

- Best / Average / Worst

```
def linearSearch(array, element):
    for i in range(0, len(array)):
        if (array[i] == element):
            return i
    return -1
```

array	0	8	6	5	9	7	2	1	4	3

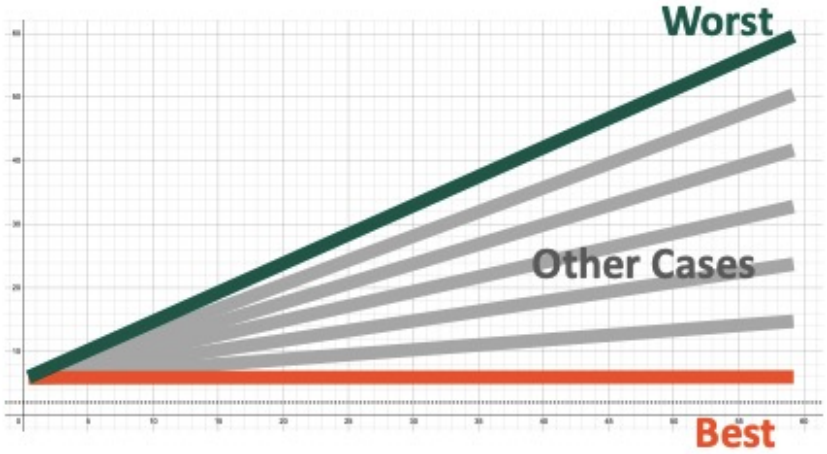
`element = 0` BEST CASE $\Omega(1)$ / $O(1)$ / $\Theta(1)$

`element = 10` WORST CASE $\Omega(N)$ / $O(N)$ / $\Theta(N)$

CASE ANALYSIS

- Best / Average / Worst

```
def linearSearch(array, element):
    for i in range(0, len(array)):
        if (array[i] == element):
            return i
    return -1
```



CASE vs ASYMPTOTIC ANALYSIS

- Straight to ASYMPTOTIC analysis
  - Why: runtime affected only by N

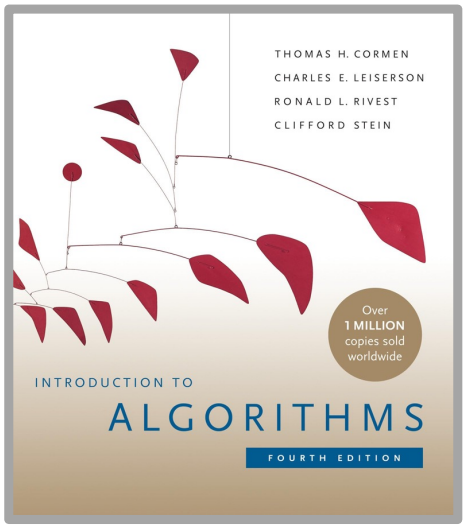
```
def isPrime(n):
    toTest = 2
    while(toTest < n):
        if (n % toTest == 0):
            return False
        else:
            toTest += 1
    return True
```

- CASE analysis first, then ASYMPTOTIC analysis
  - Why: runtime affected by something other than N

```
def linearSearch(array, element):
    for i in range(0, len(array)):
        if (array[i] == element):
            return i
    return -1
```

OUTLINE

- Topic 1 – Fundamentals
  - Recap of topics 1.1 and 1.2
  - Order of Growth (OOG)
  - Exercises on OOG
- What's next



OUTLINE

- Topic 1 – Fundamentals
  - Recap of topics 1.1 and 1.2
  - Order of Growth (OOG)
  - Exercises on OOG
- What's next

EXERCISES ON OOG

[Self-practice] Analysis of algorithms

A. Logarithms

1. $a^b = c \rightarrow b = \log_a c$ ← Definition
2. $\log_b b = 1$, $\log_b 1 = 0$ ← Special cases
3. $\log_b(\frac{x}{y}) = \log_b x - \log_b y$
4. $\log_b(x \times y) = \log_b x + \log_b y$
5. $\log_b x^y = y \times \log_b x$ ← Follows directly from the previous rule.
6. $\log_b x = \dfrac{\log_c x}{\log_c b}$ ← Changing bases
7. $x^{\log_b y} = y^{\log_b x}$
8. $b^{\log_b x} = x$ ← Follows directly from the previous rule.
9. $\lg(n!) = \sim n \lg n$ ← Stirling's Approximation

Notation:

- $\log n$ (no base) → used with orders of growth to indicate that the base is not important. Logarithms with different bases differ by a constant factor as shown in the listed identities.
- $\lg n$ → base 2.
- $\ln n$ → natural logarithm (base is e)

B. Summations

1. $\sum_{i=1}^{n} i = 1 + 2 + \ldots + n$ ← Definition
2. $\sum_{i=1}^{n} c = c + c + \ldots + c = c \times n$ ← If c does not depend on i.
3. $\sum_{i=1}^{n} c \times f_i = c \times \sum_{i=1}^{n} f_i$
4. $\sum_{i=1}^{n} f_i + g_i = \sum_{i=1}^{n} f_i + \sum_{i=1}^{n} g_i$
5. $\sum_{i=1}^{n} i = 1 + 2 + 3 + \ldots + n = \dfrac{n(n+1)}{2}$
6. $\sum_{i=1}^{n} i^2 = 1^2 + 2^2 + 3^2 + \ldots + n^2 = \dfrac{n(n+1)(2n+1)}{6}$
7. $\sum_{i=0}^{n} r^i = r^0 + r^1 + r^2 + \ldots + r^n = \dfrac{r^{n+1} - 1}{r - 1}, \quad r \neq 1$ ← Geometric Sum.
8. $\sum_{i=0}^{n} 2^i = 2^{n+1} - 1$, $\sum_{i=0}^{n} (\frac{1}{2})^i = 1 + \frac{1}{2} + \frac{1}{4} + \ldots + \frac{1}{2^n} = \sim 2$ ← Special cases of a geometric sum ($r = 2$ and $r = 0.5$).
9. $\sum_{i=1}^{n} \dfrac{1}{i} = 1 + \dfrac{1}{2} + \dfrac{1}{3} + \ldots + \dfrac{1}{n} \sim \int_1^n \dfrac{1}{i}\, \mathrm{d}i = \ln n$ ← Harmonic Number H_n.

Prepared by Ibrahim Albluwi (isma@cs.princeton.edu). Many thanks to Kevin Wayne for providing very valuable feedback.

EXERCISES ON OOG

[Self-practice] Analysis of algorithms

What is Θ of:

```
for (i=0; i<N; i++)
    for (j=0; j<N; j++)
       op();
    for (j=1; j<N; j=j*2)
       op();
```

a) N
b) N log N
c) N^2
d) N^2 + N log N

EXERCISES ON OOG

[Self-practice] Analysis of algorithms

What is Θ of:

```
for (i=0; i<N; i++)
    for (j=0; j<N; j++)
       op();
    for (j=1; j<N; j=j*2)
       op();
```

a) N
b) N log N
c) N^2
d) N^2 + N log N

EXERCISES ON OOG

[Self-practice] Analysis of algorithms

What is Θ of:

```
for (i=1; i<=N; i++)
    for (j=1; j<=i; j++)
       op();
```

a) ½ N^2
b) N log N
c) N^2 + N
d) N^2

EXERCISES ON OOG

[Self-practice] Analysis of algorithms

What is Θ of:

```
for (i=1; i<=N; i++)
    for (j=1; j<=i; j++)
       op();
```

a) ½ N^2
b) N log N
c) N^2 + N
d) N^2

EXERCISES ON OOG

[Self-practice] Analysis of algorithms

What is Θ of:

```
void f(int N) {
    if (N==0) return;
    op()
    f(N/2)
}
```

a) sqrt(N)
b) log N
c) N
d) N^2

EXERCISES ON OOG

[Self-practice] Analysis of algorithms

What is Θ of:

```
void f(int N) {
    if (N==0) return;
    op()
    f(N/2)
}
```

a) sqrt(N)
b) log N
c) N
d) N^2

OUTLINE

- Topic 1 – Fundamentals
  - Recap of topics 1.1 and 1.2
  - Order of Growth (OOG)
  - Exercises on OOG
- What's next

WHAT'S NEXT

- Topic 1.3: Abstract Data Types
- Self-practice: Data Structures
- Lab session: TA Lab – Week 2

Muddiest Point (Weeks 1 & 2 - Fundamentals)

Closes Wed 22nd Jan 2025 @ 23:55

Module Pulse Survey #1

Closes Fri 24th Jan 2025